

Python Exam Question Structures

Type 1 · Function with a Formula

e.g. Q1

The question gives formulas, and the function **returns** the result.

```
def function_name(param1, param2, param3):
    step1 = <formula using params>
    step2 = <formula using params>
    result = <formula using step1, step2>
    return result

a = float(input("Enter ...: "))
b = float(input("Enter ...: "))
c = float(input("Enter ...: "))

answer = function_name(a, b, c)
print()
print("Label:", round(answer, 2), "<unit>")
```

Checklist

- Use the **exact function name and parameter order** from the question.
- Use `float(input())` for decimals and `int(input())` for whole numbers (ages, counts).
- The function must return the result. Don't print inside it unless the question asks you to.
- Only use `round(x, 2)` if the question says "rounded to 2 decimal places".
- Write the function first, then the test code that asks for input, calls the function and prints the result.

General Exam Rules

all questions

- Put each question in its own section with a header comment.
- If the question says "define a function", write the function first, then the test code that calls it.
- Make the `input()` prompts and `print()` labels match the sample run.
- Test with the sample input, then with **another input** (it must work for any valid value).
- Don't use `sum`, `max`, `min` or `pow` unless allowed. Write the logic with loops and `if` statements.
- Allowed: `print()`, `input()`, `int()`, `float()`, `range()`, `round()`.

Type 2 · Conditions / Categories

e.g. Q2

The question gives a table of ranges plus a bonus or exception rule.

```
def function_name(value, flag):
    # 1. Special cases that skip everything else
    if <special case>:
        return <fixed value>

    # 2. Pick the category, lowest range first
    if value <= <limit1>:
        result = <amount1>
    elif value <= <limit2>:
        result = <amount2>
    else:
        result = <amount3>

    # 3. Extra rules (surcharge, discount...)
    if flag == "yes":
        result = result + <extra>

    return result

value = int(input("Enter ...: "))
flag = input("Question? ")

answer = function_name(value, flag)
print()
print("Label: $" + str(answer))
```

Checklist

- Handle the **exception first** and return early (e.g. "children under 5 are always free").
- With `elif`, each condition only needs the **upper limit**; earlier branches already caught lower values.
- Boundary words: "under 5" = `< 5`, "5–17" = `<= 17`, "65 or older" = `else or >= 65`.
- Compare text with `==` (`flag == "yes"`). Don't convert it with `int()`.
- Print money as `"Label: $" + str(answer)` so there's no space after the \$.

Python Exam Question Structures

Type 3 · Loop with Totals

e.g. Q3

The question asks for N inputs, then totals, counts, and sometimes averages.

```
total = 0.0          # running total, starts at 0
total_cost = 0.0     # second running total
count = 0            # counter, starts at 0

for i in range(1, N + 1):    # "day 1", "day 2"...
    value = float(input("Enter value for item "
                        + str(i) + ": "))

    if value <= <limit>:
        cost = value * <rate1>
    else:
        cost = value * <rate2>
        count = count + 1    # count matches

    total = total + value    # add to totals
    total_cost = total_cost + cost

print()
print("Total:", total)
print("Count:", count)
print("Total cost: $" + str(round(total_cost, 2)))
# average = total / N    (if asked for an average)
```

Checklist

- **Start every total at 0 before the loop**, never inside it.
- Add to the totals **inside** the loop; print **after** it, without indentation.
- No `sum()`, `max()` or `min()`. Add by hand: `total = total + value`.
- **Max/min by hand**: use the first value as the starting max, then `if value > biggest: biggest = value`.
- `range(1, N + 1)` counts 1 to N . `range(N)` counts 0 to $N-1$.

Type 4 · Nested-Loop Pattern

e.g. Q4

The question gives n , and you print a grid or triangle.

```
n = int(input("Enter a positive integer: "))
print()

for row in range(n):          # outer = rows
    for col in range(<width>): # inner = items
        print(<what to print>, end=" ")
    print()                   # next line
```

Shapes: swap these into the template

Pattern	<width>	<what to print>
Square of *	n	"*"
Alternating rows 0/1	n	0 if $\text{row} \% 2 == 0$ else 1
Alternating columns 0/1	n	$\text{col} \% 2$
Checkerboard 0/1	n	$(\text{row} + \text{col}) \% 2$
Triangle growing (1, 2, 3...)	$\text{row} + 1$	"*"
Triangle shrinking	$n - \text{row}$	"*"
Number triangle 1 / 1 2 / 1 2 3	$\text{row} + 1$	$\text{col} + 1$
Each row repeats its number	$\text{row} + 1$	$\text{row} + 1$

Checklist

- Use `end=" "` to keep numbers on one line, and an empty `print()` to end each row.
- $\text{row} \% 2 == 0$ is true for even rows (0, 2, 4...), so use it for anything that alternates.
- Check your loop with the smallest case, $n = 1$ or 2.